Written Homework 2: Computer Organization

CS 255: Computer Architecture and Machine Level Programming

Name: FELIPE CARDOZO NetID: 2676984

- **Objective**: This assignment provides guided practice reasoning about C programming concepts.
- **Submission**: Upload a completed copy of this PDF to Gradescope.
- **Grading**: Each problem is 10 points and will be graded using a check system. Showing your work is strongly encouraged.
  - Check-plus (100%): Correct solution with clear and complete reasoning.
  - Check (90%): Mostly correct, with minor error(s) or omission(s).
  - Check-minus (70%): Partially correct, with demonstrated understanding but significant mistakes or incomplete work.
  - No credit (0%): No meaningful attempt or a major conceptual misunderstanding.

1. *[Computer Overview]* Consider the following C program:

```
int x;
int y;

scanf("%d", &x);
y = x + 10;
printf("%d\n", y);
```

For each event below, identify the main component(s) involved and describe what happens to the data.

Event	**Component(s)**	**What happens to the data?**
the user enters 25	I/O device	Keyboard sends input 25 into computer
`25` becomes the value of `x`	CPU & Memory (RAM)	CPU process input and stores integer value 25 in memory location for x
`x` is obtained for the computation	CPU & Memory (RAM)	CPU reads value 25 from x's location in RAM into the CPU
`x + 10` is computed	CPU (ALU)	ALU computes 25+10 to produce 35
the result becomes the value of y	CPU & Memory (RAM)	CPU stores result 35 in memory location for y
the value of y is displayed	CPU, Memory (RAM), and I/O device	CPU reads y (35) from RAM and sends it to the monitor/terminal, displaying it

2. *[Binary Representation]* Consider the 8-bit binary string `10110110`.

(a) Give the decimal value of the string when interpreted as an unsigned integer.

10110110 → 182
128 32 16 4 2

(b) Give the decimal value of the string when interpreted as a two's-complement signed integer.

10110110
01001001 +1
-01001010 = -74
64 8 2

10110110
-128 +64 +8 +2 = -74

(c) Write the string in hexadecimal.

1011 0110
B 6

1011 → 11 ≡ B
0110 → 6 ≡ 6

10110110_2 = 0xB6

(d) Sign-extend the string to 16 bits.

11111111 10110110

sign bit → 0 → extend 0
sign bit → 1 → extend 1

(e) What decimal value does your answer to part (d) represent as a 16-bit two's-complement integer? Why?

extending 0's turns original negative number into 182 > 0

$-32768 + \sum_{0}^{13} 2^n - 1 - 8 - 126 = -74 \equiv 11111111\ 10110110$

sign extension copies the original sign bit (1) into new leftmost bits which preserves the value of the negative number

0xFFB6 = 65462
$65462 - 2^{16} = -74$

3. *[Binary Representation]* Perform each of the following operations in binary. Show your binary arithmetic, the resulting 8-bit binary value and its decimal interpretation, and state whether overflow occurs and why. Assume all values are 8-bit two's-complement signed integers.

(a) `00110110 + 00011101`

00110110
00011101

64 + 29 = 83

$01010011_2 = 83$

No overflow, both operands are positive and result is positive within [-128, 127] (8-bit signed range)

(b) `01100101 + 00101100` = 10010001 = -128 + 16 + 1
= -111

Overflow: 101 + 44 = 145 > 127
↳ adding two positive values produced a negative bit pattern

(c) `10110110 + 11100101`

-128 + 32 + 16 + 4 + 2 = -74 ↘ -128 + 64 + 32 + 4 + 1 = -27

∴ -128 ≤ -101 ≤ -127, result in [-128, 127], Hence no overflow

10110110
+ 11100101

$\nwarrow 1001\,1011_2 = -101$

4. *[Real Numbers]* Consider a 8-bit floating point representation with:

   - 1 sign bit,
   - 4 exponent bits (using excess-7 encoding), and
   - 3 fraction bits.

   Assume that this data type uses the conventions presented in class, including representations of zero, NaN, and (+/-)infinity.

   S | EEEE | MM

(a) Represent the decimal value 0.8125 in this representation.

$.8125 = .1101_2 = 1.101_2 \cdot 2^{-1}$ (4)

sign bit $_2 = 0$

exponent $e = -1$

stored exponent $E = e + 7 = 6 = 0110_2$

0 | 0110 | 101

00110101

(b) What decimal value does the bit string 1|1010|011 represent?

$e = 2 = 1$

$E = 10$

$e = E - 7 = 3$

$1.011_2 = 1 + 0/2 + 1/4 + 1/8 = 1.375 \quad \therefore (-1)^1 * 1.011_2 * 2^3 = -1.375 * 8 = -11$

component	bias	purpose
sign	1	determines positive or negative
exponent	4	determines power of 2
fraction	3	determines significand
exponent bias	7	

can represent
1, 1.125, 1.25

5. *[Real Numbers]* Answer the following questions using the same floating point representation as the previous question.

(a) What is the largest positive representable value, other than infinity, of this representation?

$x = (-1)^S (1 + F) 2^{E-7}$

0 1110 111
sign exp frac

↳ largest 1111=15, 0 to 15 infinity → 1110 = 14

$E = 1110_2 = 14 \rightarrow e = 14 - 7 = 7$

sign 0: largest frac 111 → $1.111_2 = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} = \frac{15}{8}$, $\left(\frac{15}{8}\right) 2^7 = 240$

(b) What is the smallest positive representable value, other than zero, of this representation?

0 | 000 | 001 $\quad 0.001_2 = 1/8$
E=1

exponent $e = 1 - 7 = -6$

$0.001_2 = 1/8 \cdot 2^{-6} = 2^{-9} = 1/512$

(c) What is the step size (difference between adjacent representable values) between 1.0 and 2.0?

Step size $= 1.001_2 - 1 = \frac{9}{8} - 1 = \frac{1}{8}$

fraction has 3 bits, the least significant fraction bit $2^{-3} = 1/8$

$1.001_2 - 1.000_2 = 1.125 - 1 = 1/8$

6. *[Memory]* Consider the following declarations:

```
struct Record {
    short values[3];
    int id;
    char flag;
};
struct Record records[4];
```

array 6 bytes ↔ 3 * sizeof(short) = 3 * 2 = 6
int 4 bytes
character 1 byte
11 bytes + padding which is divisible by 4

(a) Draw the memory layout of `records[0]`, clearly indicating padding and the end of the struct. Assume that `&records = 0x1000`.

values[0] 0x100 - 0x1001
values[1] 0x102 - 0x1003
values[2] 0x104 - 0x1005
Padding 0x1006
Padding 0x1007
id 0x1008 - 0x100B
flag 0x100C
Padding 0x100D
Padding 0x100E
Padding 0x100F

(b) What is the starting address of `records[2]`?

&records[2] = 0x1000 + 2 (16) = 0x1020
↓ records[0]
↘ Struct Record

$16_{10} = 0x10$

(c) What is the address of `records[2].values[1]`?

0x1020 + 0x2
0x1020

Addr(records[2].values[1]) = 0x1000 + 16·2 + 2·1
= 0x1022

2^{16} distinct addresses, each address refers to 1 byte

7. *[Memory]* Consider a computer system with a 16-bit address bus and a 32-bit data bus.

   (a) What is the maximum byte-addressable memory capacity?

Max Capacity: $2^{16} \cdot 1\text{ byte} = 65,536\text{ bytes} \sim 64\text{ KiB}$

→ 1 memory access

   (b) Suppose the CPU reads a 4-byte int beginning at address 0x1008. How many memory accesses are required? What about a 4-byte int beginning at address 0x100A? Explain.

↳ 2 memory access

32-bit transfers 4 bytes per memory access

↳ start at 0x1008, 0x1009, 0x100A, 0x100B (1 memory access)

↳ start at 0x100A, 0x100B, 0x100C, 0x100D (2 memory access)

A 4-byte integer requires 1 access if it fits within a single 4 byte block otherwise, requiring 2 access

   (c) Suppose instead that the computer has a 16-bit data bus. How many memory accesses would be required to read an aligned 4-byte int? How does this compare with the original system?

16 bit data bus transfer: $\frac{16\text{ bits}}{8\text{ bits/byte}} = 2\text{ bytes/access}$ ∴ Memory access = $4/2 = 2$

(int occupies 4 bytes)

Reducing the databus from 32 to 16-bits doubles the # memory access needed to read the 4-byte integer

8. *[CPU Structure]* An 8-bit register `status` stores several values using the following format:

mode	A	B	C
Bits: 7--3	2	1	0

The five-bit field `mode` represents an unsigned integer from 0 to 31.

Write a C expression for each of the following.

(a) Determine whether flag `B` is set.

(status & (1<<1)) != 0) ≡ (status & 0x02) != 0 → result is not 0, so B is a set

Flag B is stored at bit position 1

1<<1 creates mask 00000010

& isolates bit 1

status: 10110110

mask: 00000001

result: 10 ≠ 0

(b) Set flag `A`.

status |= (1<<2);

bit 2 set, creates mask

(c) Extract the value of `mode` as an integer from 0 to 31.

mode occupies bits 7-3 (status >> 3) & 0x1F

shifts bits 7-3 into position 4-0

Keeps only lowest 5 bits (00011111)

Status: 10110010

Shifted: 00010110

mask: 00011111

result: 00010110 = 22

∴ Mode 22

↗ 15.pdf (12-19)

9. *[Instruction Execution Cycle]* Consider the following instruction which shifts the value in `R2` left by 2 bits and stores the result in `R1`.

CPU retrieves instruction `shift R1, R2, 2` from memory

Describe what happens during each stage of the instruction execution cycle:

(a) Fetch

CPU places current PC value on the address line, sending memory-read signal. Memory returns shift $R_1, R_2, 2$ over the data bus to control unit

(b) Decode

Control unit interprets the instruction, performing Left shift, read source register R_2 write destination register R_1, and use immediate shift amount 2. It selects next PC value PC+4, and disables data-memory access

(c) Execute

The CPU reads value in R_2 and sends it into ALU/shifter, which shift the value left by 2 bit positions, getting $R_2 << 2$

(d) Memory

No data-memory operation occurs because this instruction only operates on registers

(e) Writeback

The CPU writes the shifted result from ALU/shifter into R_1

p 151(27-36)

10. *[Instruction Execution Cycle]* A program executes 600 million instructions on a 3 GHz processor with an average CPI of 2.0.

(a) How long does the program take to execute?

$$\frac{600\text{M} \cdot 2.0 \text{ cycles/instructions}}{3\text{B cycles/s}} = \frac{12 \cdot 10^8}{3 \cdot 10^9} = .4 \text{ s}$$

(b) A processor improvement reduces the average CPI to 1.0, but requires the clock rate to decrease to 2 GHz. How long does the program take to execute on the improved processor?

$$\frac{600\text{M} \cdot 1.0 \text{ cycles/instructions}}{2\text{B cycles/s}} = \frac{6 \cdot 10^8}{2 \cdot 10^9} = .3 \text{ s}$$

(c) Calculate the speedup from the improvement.

$$\text{Speedup} = \frac{\text{Og execute } \Delta t}{\text{improved exe } \Delta t} = \frac{.4}{.3} = 1.33 \text{ times faster}$$

(d) Explain why reducing CPI does not necessarily reduce execution time.

$$\text{Execution } \Delta t = \frac{\text{instruction count} \cdot \text{CPI}}{\text{clock rate}}$$

execution time depends on clock rate. A lower CPI can offset by lower clock rate. In the problem, there was improvement because clock rate decreased enough to still allow improvement even with the lower CPI

16 (8-11)

11. *[Caches]* A designer is considering a cache hierarchy with an L1 cache with a 25% miss rate and 1 cycle access time backed by a larger L2 cache with a 7% miss rate and 5 cycle access time. The L2 cache is backed by memory with a 100 cycle access time.

    (a) What is the average access time, in cycles, of the L2 cache?

$t_{L2} = t_{L2\,hit} + L_{2\,miss\,rate} * t_{memory}$

$5 + (.07 * 100) = 12$ cycles

    (b) What is the average access time, in cycles, of the L1 cache?

$t_{L1} = t_{L1\,hit} + L_{1\,miss\,rate} * t_{L2}$

$= 1 + .25 * 12 = 4$ cycles

    (c) Suppose the L2 cache is removed, so the L1 cache is backed directly by main memory. How much slower would the average memory access time be?

$t_{NoL2} = t_{L1\,hit} + L_{1\,Miss\,rate} * t_{memory}$

$= 1 + .25 * 100 = 26$ cycles

additional delay $= t_{NoL2} - t_{L1}$

$= 26 - 4 = 22$ cycles

∴ How Much slower $\frac{t_{NoL2}}{t_{L1}} = \frac{26}{4} = 6.5$ times slower

16 (13 – 23)

12. *[Caches]* Consider a byte-addressable system with 12-bit addresses and a 64-byte direct-mapped cache. Each cache block holds 8 bytes.

(a) How many blocks can the cache hold?

$$\text{Cache Blocks} = \frac{\text{cache size}}{\text{block size}} = \frac{64 \text{ bytes}}{8 \text{ bytes/block}} = 8 \text{ blocks}$$

(b) How many bits of a memory address are used for the byte offset, set index, and tag?

offset: $\log(8 \text{ bytes/block}) = 3$ bits

set index: $\log(8 \text{ sets}) = 3$ bits w/ 1 block per set → 8 sets

Tag: $12 - 3 - 3 = 6$ bits

(c) Determine the tag, set index, and outcome for each memory access. Assume the cache is initially empty.

Address	Tag	Set Index	Hit/Miss
0 0 0 0 0 0 | 0 0 0 | 0 0 0	000000	000	Miss
0 0 0 0 0 0 0 1 1 0 0 0	000000	001	Miss
0 0 0 0 0 0 0 0 0 0 1 1	000000	000	hit
0 0 0 0 0 1 0 0 0 0 0 0	000001	000	Miss
0 0 0 0 0 0 0 0 0 0 0 0	000000	000	Miss
0 0 0 0 0 0 0 1 1 0 0 0	000000	001	hit